EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Frameworks, tools, tracing, and testing

Instructor:
Arash Saifhashemi

UNIVERSITY OF SOUTHERN CALIFORNIA

EE-599: Designing and Building Autonomous AI Agents

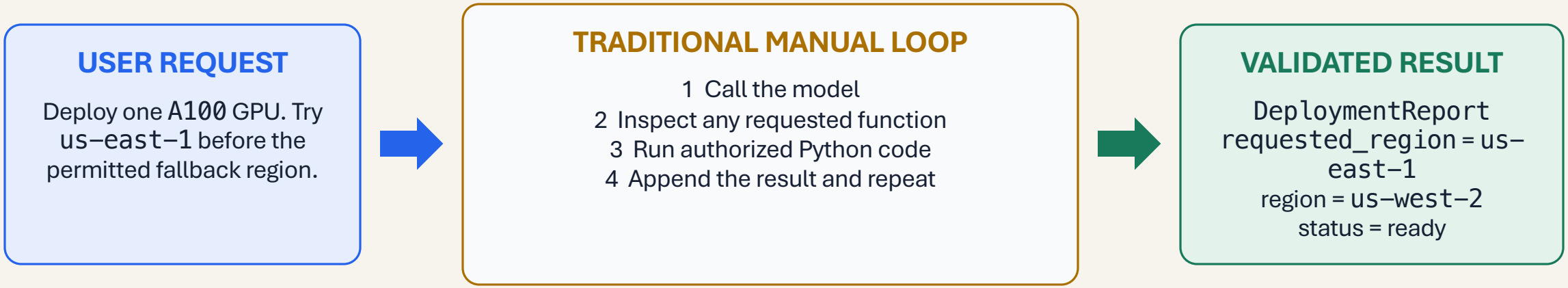
Session 02

Frameworks, tools, tracing, and testing

Graduate engineering lecture deck

The application goal and the traditional implementation

Goal: turn one deployment request into one authorized cloud change and a validated `DeploymentReport`.



WHAT TRADITIONAL APPLICATION CODE DOES

The application calls the model, reads any requested function call, validates and executes authorized Python code, appends the result to the model input, and repeats until the model supplies a final answer.

Traditional application code implements the repeated model-and-tool process directly.

Why the manual loop becomes difficult to operate

Session focus: reliable software for model-driven actions. Prompt design is outside today's scope.

QUESTIONS THE MANUAL LOOP MUST ANSWER

- How long may model and tool calls wait?
- Which failed operations may run again?
- How do approval decisions and errors reach the next model call?
- How does one request avoid creating the same cloud resource twice?
- How many model calls may occur before the application stops?

WHAT THIS SESSION TEACHES

RUNNER

The SDK execution engine coordinates repeated model calls and host function calls.

FUNCTIONTOOL AND PYDANTIC

Python functions become model-visible definitions with validated arguments and typed final output.

STREAM EVENTS AND TRACE SPANS

Live events report progress; timed operation records support diagnosis after a run.

PYTEST AND SCRIPTEDMODEL

Direct function tests and deterministic model responses verify different parts of the workflow.

The SDK coordinates the repeated process. Application code controls real cloud actions.

What you will learn

By the end of this session, you should be able to:

1

EXPOSE MISSING OPERATING RULES

Explain why a working model-and-tool loop still needs explicit waiting, retry, approval, replay, and stopping policies.

2

ASSIGN EXECUTION RESPONSIBILITIES

Separate what `Agent` and `Runner` coordinate from the authorization and cloud behavior that application code owns.

3

FOLLOW ONE TYPED DEPLOYMENT

Trace the canonical `CapacityReport` and `DeploymentReport` through model calls, tool execution, approval, and final validation.

4

MATCH EVIDENCE TO THE QUESTION

Use stream events and traces to inspect a run, then choose a unit test, deterministic workflow test, evaluation, or integration test.

The same GPU deployment request connects every outcome to one concrete system.

A model can request an action; application code must execute the action

A USER ASKS FOR A REAL INFRASTRUCTURE CHANGE

USER REQUEST

Deploy one `A100` GPU. Use `us-west-2` only if `us-east-1` has no capacity.

MODEL OUTPUT · unverified function-call proposal

```
{
  "type": "function_call",
  "name": "check_capacity",
  "arguments":
    "{\"region\":\"us-east-1\",\"gpu_count\":1}"
}
```

The model proposes a capacity check. The host must validate and execute it before any fallback or mutation is justified.

HOST APPLICATION

Application code still has to:

- Validate the arguments supplied by the model
- Check authorization and approval
- Call Python code and the cloud service
- Return the result and decide whether to continue

An agent application needs an execution loop around each model response.

Seven sections follow one deployment request.

1	RAW LOOP	Why does a minimal loop still need timeout, model-visible failure, duplicate-action prevention, and stopping rules?
2	PLATFORM	Where do the Agents SDK, model service, hosted tools, and application functions run?
3	RUNNER	How does the SDK execution engine coordinate several model calls and host tool calls?
4	TOOLS	How do Python functions, docstrings, type hints, schemas, and `Pydantic` validation work together?
5	OBSERVE	How do live stream events and timed trace spans expose the operations in one run?
6	TEST	What should `pytest`, `ScriptedModel` workflow tests, and evaluations that use a real model verify?
7	COMPLETE APP	How do `Agent`, `Runner`, `FunctionTools`, `output_type`, approval, and tests fit into one program?

Every section returns to the same request: deploy one GPU safely and return usable data.

Section 1: Why the raw loop is insufficient

Use one GPU deployment request to identify the timeout, model-visible failure, duplicate-action prevention, and stopping decisions missing from the smallest working loop.

The deployment request, allowed fallback, and required result

HYPOTHETICAL GPU DEPLOYMENT AGENT

USER REQUEST

Deploy one `A100` GPU in `us-east-1`. If it is full, application policy permits `us-west-2` as the only fallback.

ONE RUN · one `Runner` execution

One run starts with this user request. It may contain several model turns, meaning separate model invocations, with host-side tool execution between them.

PROTOCOL EXAMPLE · final output

```
{
  "instance_id": "srv-a7",
  "requested_region": "us-east-1",
  "region": "us-west-2",
  "status": "ready"
}
```

SUCCESS MEANS ALL THREE CONDITIONS ARE TRUE

- Exactly one server is created
- Every tool call is inspectable
- `Runner` returns a validated `DeploymentReport`

Success requires one authorized deployment, inspectable operations, and a validated final object.

A minimal loop repeats model calls and executes requested functions

MISSING RULE

How long may a model request wait or retry?

```
ABRIDGED APPLICATION CODE · week_1_loop.py

while True:
    response = call_model(history, tools)
    if response.tool_calls:
        outputs = execute_tools(response.tool_calls)
        history.extend(outputs)
    else:
        return response.text
```

MISSING RULE

What happens when validation or tool code fails?

MISSING RULE

How many model calls are allowed before stopping?

history: the user, model, and `tool-result` items accumulated so far

tools: application functions described to the model

The loop handles the success path, but it leaves failure and stopping behavior unspecified.

The manual loop leaves four different questions unanswered

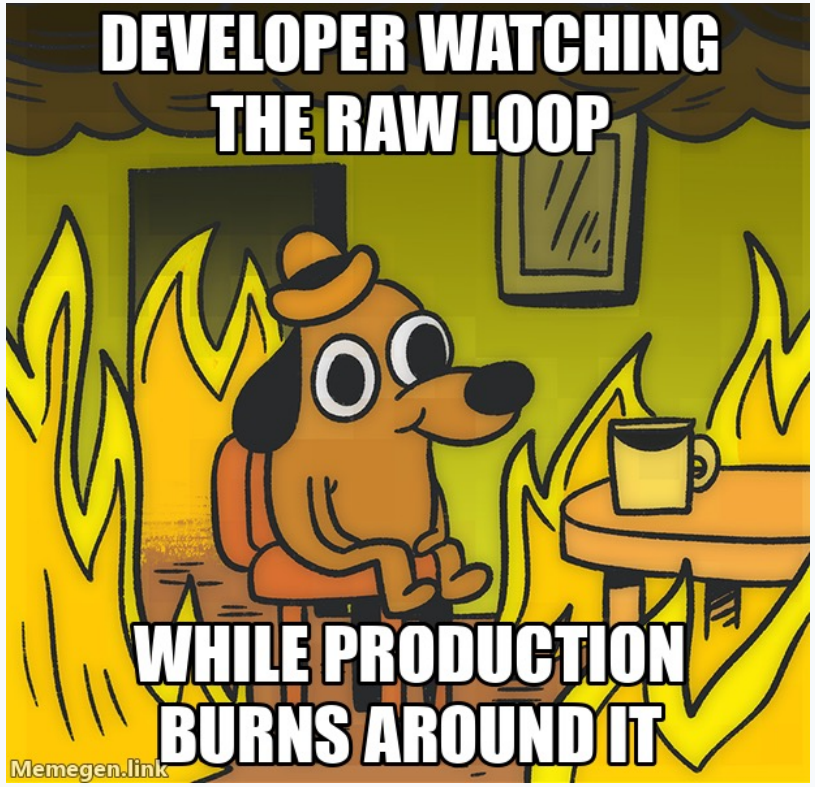
EACH QUESTION GOVERNS A DIFFERENT FAILURE OR DELAY

DECISION	QUESTION THE APPLICATION MUST ANSWER	WHAT CAN HAPPEN IF NO RULE EXISTS
WAITING	**How long may one model call or tool call wait?**	A slow dependency can occupy the request worker indefinitely.
RETRYING	**Which failed calls may run again, and under what conditions?**	The run may abort on a temporary failure or repeat a side effect.
STOPPING	**How many model invocations may one application run make?**	Repeated tool requests can continue adding latency and model cost.
RECOVERING	**Who receives an error, and which saved state may resume?**	The process may crash or lose the work completed before the failure.

Together, these rules determine how the application behaves when the success path breaks.

Three failures can stop the deployment request

POSSIBLE FAILURE MODES IN THE GPU DEPLOYMENT APPLICATION



UNHANDLED TOOL EXCEPTION

Trigger: the capacity client raises an unexpected exception.
Consequence: the run ends without a final value, and an API endpoint may return HTTP 500.

REPEATED MUTATION

Trigger: a lost acknowledgement causes `provision_gpu` to run again.
Consequence: one user request creates two servers and duplicate cost.

UNUSABLE FINAL OUTPUT

Trigger: the model returns prose without the required fields.
Consequence: application code cannot read `instance_id`, region, or status.

The meme summarizes the risk. Each explanation names a specific trigger and consequence.

Timeouts, retries, and turn limits answer different questions

CONTROL	CONFIGURATION	QUESTION ANSWERED	OPERATIONAL EFFECT
Model-call timeout	`ModelSettings(timeout=30.0)`	How long may one model-call attempt wait?	A slow attempt ends after the configured time.

A model-call timeout limits one attempt, but it does not limit the complete run.

Timeouts, retries, and turn limits answer different questions

CONTROL	CONFIGURATION	QUESTION ANSWERED	OPERATIONAL EFFECT
Model-call timeout	`ModelSettings(timeout=30.0)`	How long may one model-call attempt wait?	A slow attempt ends after the configured time.
Model retry policy	`ModelSettings(retry=...)`	May a failed model call be attempted again?	Retries are opt-in and need replay-safe inputs.

A retry policy separately decides whether another model-call attempt is permitted.

Timeouts, retries, and turn limits answer different questions

CONTROL	CONFIGURATION	QUESTION ANSWERED	OPERATIONAL EFFECT
Model-call timeout	`ModelSettings(timeout=30.0)`	How long may one model-call attempt wait?	A slow attempt ends after the configured time.
Model retry policy	`ModelSettings(retry=...)`	May a failed model call be attempted again?	Retries are opt-in and need replay-safe inputs.
Tool timeout	`@function_tool(timeout=15.0)`	How long may an asynchronous tool invocation wait?	The timeout can become model-visible feedback or an exception.

A tool timeout limits one asynchronous host function call and needs its own failure policy.

Timeouts, retries, and turn limits answer different questions

CONTROL	CONFIGURATION	QUESTION ANSWERED	OPERATIONAL EFFECT
Model-call timeout	`ModelSettings(timeout=30.0)`	How long may one model-call attempt wait?	A slow attempt ends after the configured time.
Model retry policy	`ModelSettings(retry=...)`	May a failed model call be attempted again?	Retries are opt-in and need replay-safe inputs.
Tool timeout	`@function_tool(timeout=15.0)`	How long may an asynchronous tool invocation wait?	The timeout can become model-visible feedback or an exception.
Run turn limit	`Runner.run(..., max_turns=4)`	How many model invocations may one run make?	`Runner` stops if no final output appears in time.

A turn limit caps the total number of model invocations inside one `Runner` execution.

Agents SDK roles: configuration, execution, tools, observability, results

- `Agent`: configuration containing instructions, tools, model, and final-output type
- `Runner`: execution engine that repeats model calls, invokes tools, and stops the run
- `FunctionTool`: wrapper that describes a Python function and validates its arguments
- Approval, streaming, and tracing: pause actions, report progress, and record operations
- `RunResult`: completed-run object containing `final_output` and run information

Application code still owns cloud credentials, authorization, business rules, replay protection, and product-specific recovery.

`RunResult` gives application code the validated final value and information about the completed run.

The SDK runs the loop while application code performs the action

AGENTS SDK OWNS

- Repeated model calls inside one run
- Tool lookup, input validation, and invocation
- Handoffs, streaming events, traces, and run results
- Stopping at final output, approval, or configured limits

YOUR APPLICATION OWNS

- Cloud credentials and authorization
- The code that reads or changes infrastructure
- Idempotency, approvals, and domain rules
- Storage, HTTP responses, UI updates, and operational alerts

The SDK packages orchestration. It does not replace application security or cloud logic.

Section 2: The Agents SDK in the application

Locate the SDK, model service, and application functions, then define `Agent` and `Runner` before following an execution.

The SDK runs locally; models and hosted tools run on the OpenAI Platform.

OPENAI PLATFORM

MODEL SERVICE

HOSTED TOOLS

Examples: web search, file search

MODEL REQUEST

MODEL RESULT

YOUR APPLICATION PROCESS

AGENTS SDK / RUNNER

Runs the loop and invokes registered tools.

FUNCTION TOOLS

Your Python functions and local dependencies.

PRODUCT CODE

Authorization, storage, UI, and cloud clients.

GPU deployment agent: implemented features and previewed SDK extensions

IMPLEMENTED IN GPU DEPLOYMENT AGENT

- One `Agent` configuration
- `Runner` with several model turns
- Local `FunctionTools` and approvals
- `Pydantic` tool and final-output validation
- Streaming, tracing, and deterministic tests

INTRODUCED, NOT IMPLEMENTED HERE

Handoffs	Transfer control to another agent.
Sessions	Persist conversation state across runs.
Guardrails	Check input, output, or tool activity.
Hosted / MCP	Use Platform-managed or remote tool services.

This session builds the left-column path. Later sessions add the right-column features.

In the Agents SDK, `Agent` stores configuration that `Runner` reads.

The `Agent` object does not execute the loop. It packages the settings used on each model invocation.

ABRIDGED APPLICATION CODE · cloud_agent.py

```python
cloud_agent = Agent(
    name="Cloud deployer",
    instructions="Check capacity before provisioning.",
    model=MODEL_NAME,
    tools=[check_capacity, provision_gpu],
    output_type=DeploymentReport,
)
```

`name`

Human-readable label used in traces and orchestration.

`instructions`

System-level guidance sent with each model request.

`model`

The model implementation `Runner` invokes during each turn.

`tools`

The registered capabilities the model may request.

`output_type`

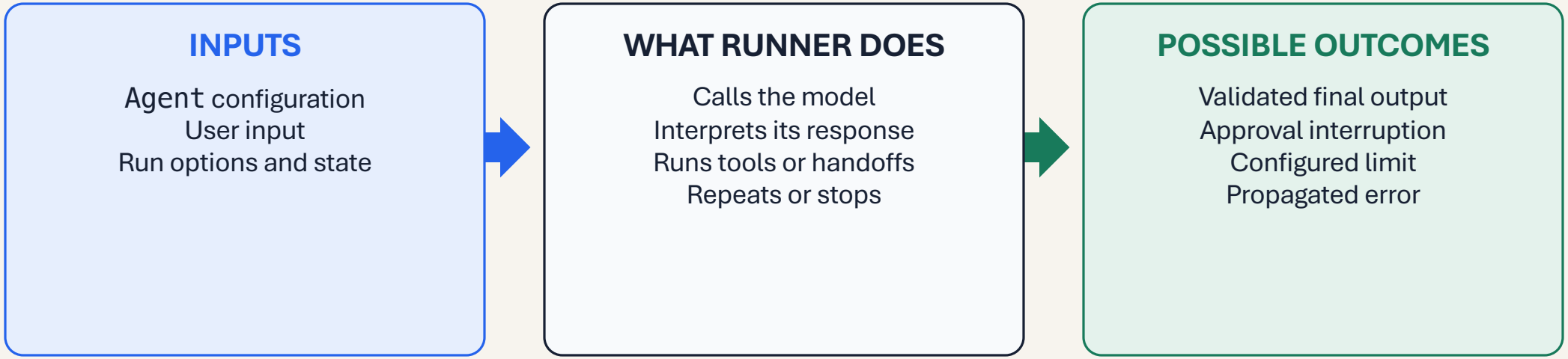
The Python type required for the completed run's result.

`Agent` supplies configuration. `Runner` uses it to perform the model-and-tool execution.

`Runner` is the SDK execution engine for one agent run.

ABRIDGED APPLICATION CODE · application code

```python
result = await Runner.run(cloud_agent, request, max_turns=4)
```



`Agent` supplies configuration. `Runner` performs repeated model calls and invokes the registered tools.

A run can return a validated `DeploymentReport` to application code

APPLICATION CODE · abridged SDK structure

```python
@function_tool
def check_capacity(region: str) -> int:
    return capacity_client.read(region)

cloud_agent = Agent(
    instructions="Check capacity before provisioning.",
    tools=[check_capacity],
    output_type=DeploymentReport,
)

result = await Runner.run(cloud_agent, request)
report: DeploymentReport = result.final_output
```

FUNCTION TOOL

A Python function the model may request. The host application still executes the code.

AGENT

Configuration that names the instructions, available tools, model, and final-output type.

RUNNER

Execution engine that repeats model calls, validates tool requests, invokes tools, and stops.

FINAL OUTPUT

The validated application value returned after the run completes.

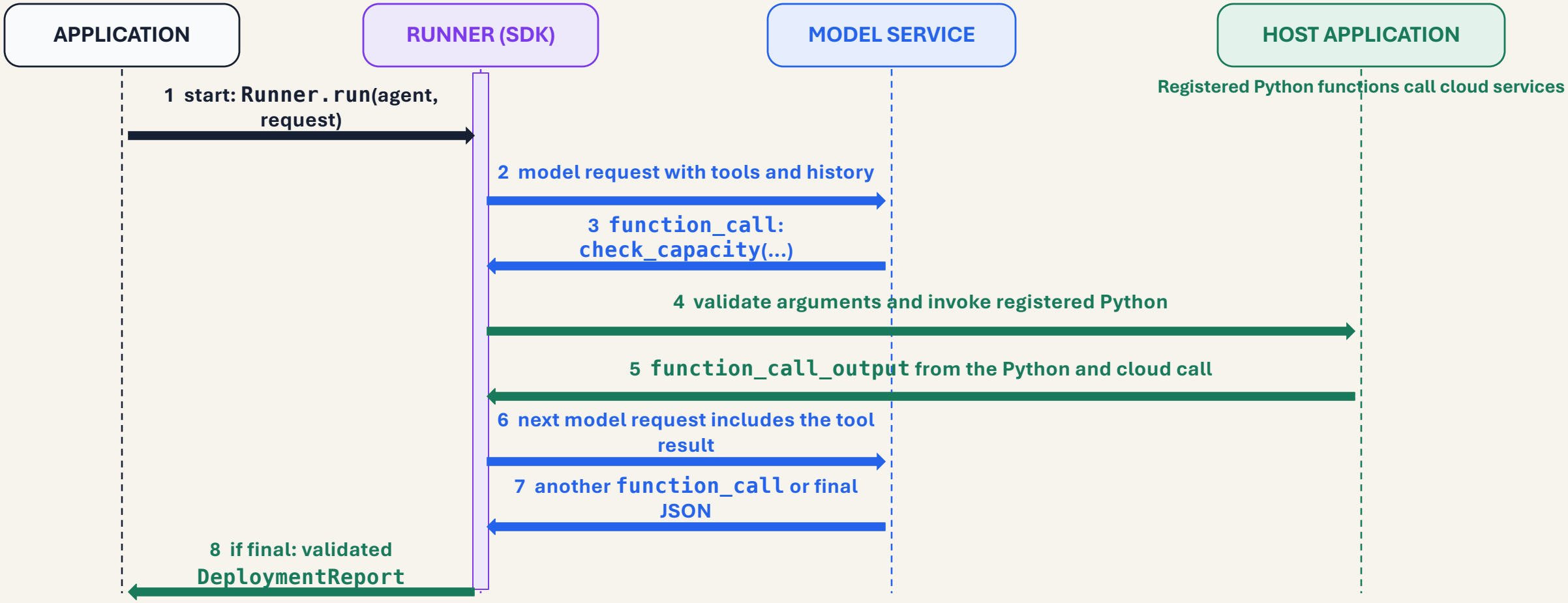
The SDK replaces manual orchestration, not the application function or its security rules.

Section 3: How `Runner` executes one request

Follow one GPU deployment request through model calls, tool results, approval, and validated final output.

One `tool-call` cycle shown as a UML sequence diagram

Time moves downward. Each horizontal arrow represents one request or returned value.



`Runner` repeats this cycle until the model returns a final value that passes validation.

A user request can contain several model turns.

ONE RUNNER EXECUTION FOR ONE APPLICATION REQUEST

MODEL TURN 1

One model invocation returns a capacity tool call.

MODEL TURN 2

A second invocation sees capacity output and requests provisioning.

MODEL TURN 3

A third invocation sees the provision result and returns final output.

`max_turns=3` counts these three model invocations. Tool execution occurs between them and is not a separate model turn.

Model turn 1 requests capacity because cloud state is external

INPUT TO THE FIRST MODEL INVOCATION

User: Deploy one `A100` in `us-east-1`. If that region is full, use an approved alternative.

Available tool description:
`check_capacity`(region, `gpu_count`)

model output item

```
{
  "type": "function_call",
  "name": "check_capacity",
  "arguments": "{\"region\":\"us-east-1\",\"gpu_count\":1}",
  "call_id": "call_cap"
}
```

`call_id` is a correlation key. `Runner` will copy it onto the matching `tool-result` item.

Interpretation: the model requests a local function call. No Python function has run yet.

A tool call is structured model output, not proof that the action happened.

Between model turns, the SDK validates and the host executes.

1 PARSE

Runner reads the function_call item and its JSON arguments.

2 VALIDATE

The generated Pydantic parameter model checks names, types, and constraints.

3 DISPATCH

Runner matches check_capacity to the registered FunctionTool.

4 EXECUTE

Your Python function calls the capacity service and returns data.

HOST OUTPUT · CapacityReport appended by Runner

```
{
  "requested_region": "us-east-1",
  "available": false,
  "available_slots": 0,
  "recommended_region": "us-west-2",
  "recommended_available_slots": 8
}
```

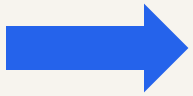
Pydantic validation occurs before your function receives model-supplied arguments.

Model turn 2 can act on tool output added by the `Runner`.

NEW INFORMATION IN THE SECOND MODEL INVOCATION

HOST OUTPUT · CapacityReport from check_capacity

```
{
  "requested_region": "us-east-1",
  "available": false,
  "available_slots": 0,
  "recommended_region": "us-west-2",
  "recommended_available_slots": 8
}
```



CONCEPTUAL PROTOCOL · decoded arguments

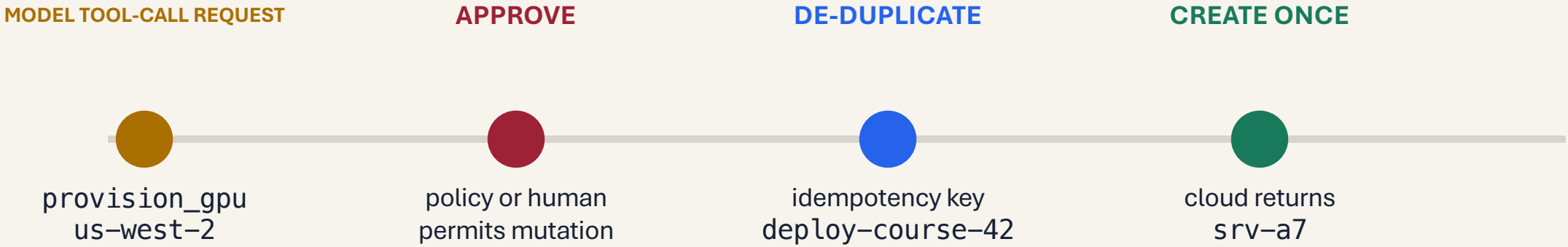
```
{
  "type": "function_call",
  "name": "provision_gpu",
  "decoded_arguments": {
    "region": "us-west-2",
    "gpu_count": 1,
    "idempotency_key": "deploy-42"
  },
  "call_id": "call_provision"
}
```

The model chose `us-west-2` because the prior tool output reported eight available slots there.

Tool output becomes input to the next model invocation inside the same run.

A mutating tool request needs authorization and replay protection.

Idempotency key: an application-generated request identifier reused on retries so the cloud service returns the first result instead of creating another server.



ABRIDGED EXAMPLE · host-side return value

```
ProvisionResult(
  instance_id="srv-a7", region="us-west-2", status="ready")
```

The model may request a mutation, but the host application decides whether and how it runs.

Sensitive tools can require approval before their Python code runs.

ABRIDGED APPLICATION CODE · cloud_tools.py

```python
@function_tool(needs_approval=True)
def provision_gpu(
    region: str, gpu_count: int, idempotency_key: str
) -> ProvisionResult:
    return cloud.create_once(
        region, gpu_count, idempotency_key)
```

`needs_approval=True` tells `Runner` to interrupt when this tool is requested, before invoking the function.



MODEL REQUEST

`provision_gpu(region="us-west-2", gpu_count=1, key="deploy-42")`



PAUSE

`Runner` returns an interruption. Tool code has not executed.

THE HOST APPLICATION REVIEWS THE PROPOSED ACTION

For the GPU deployment example, application policy permits one `A100` in `us-west-2` and rejects every other fallback region. A human or policy service can approve or reject the exact pending tool call.

Approval is a host decision about a pending action; it is not another model opinion.

Option 1: A host policy callback pre-approves safe tool calls

ABRIDGED APPLICATION CODE · openai-agents 0.22.0

```python
async def should_require_approval(ctx, args: dict, call_id: str) -> bool:
    # Programmatic host policy:
    # Allow 1 GPU in "us-central1" automatically without human intervention
    if args.get("region") == "us-central1" and args.get("gpu_count", 0) <= 1:
        return False  # Pre-approved by host; run immediately

    return True  # Exceeds safe threshold; require explicit approval

@function_tool(needs_approval=should_require_approval)
def provision_gpu(
    region: str, gpu_count: int, idempotency_key: str
) -> ProvisionResult:
    return cloud.create_once(region, gpu_count, idempotency_key)
```

False runs the call immediately; True interrupts the run for an approval decision.

Option 2: The host decides on returned interruptions, then resumes the run

ABRIDGED HOST CODE · programmatic interruption decisions

```python
import json

result = await Runner.run(cloud_agent, request)

while result.interruptions:
    state = result.to_state()
    for item in result.interruptions:
        # Check host conditions in code
        is_target = item.agent.name == cloud_agent.name and item.tool_name == "provision_gpu"
        if is_target:
            args = json.loads(item.arguments or "{}")
            if args.get("gpu_count", 0) <= 2:
                state.approve(item)  # Host auto-approves
            else:
                state.reject(
                    item, rejection_message="GPU quota exceeded host limit."
                )
        else:
            state.reject(item)

    # Resume the run with the host decisions applied
    result = await Runner.run(cloud_agent, state)
```

Model turn 3 produces the final value after the tool result is available.

LEFT: HOST RESULT THAT RUNNER ADDS TO MODEL TURN 3

HOST OUTPUT · provision_gpu result

```
{
  "call_id": "call_provision",
  "instance_id": "srv-a7",
  "region": "us-west-2",
  "status": "ready"
}
```



MODEL OUTPUT · DeploymentReport JSON

```
{
  "instance_id": "srv-a7",
  "region": "us-west-2",
  "status": "ready",
  "requested_region": "us-east-1"
}
```

`Runner` returns `result.final_output` as a `DeploymentReport` object, not as an untyped sentence.

The run ends only after the configured final-output contract is satisfied.

One request produced three model turns and two host-side tool executions.

1 MODEL TURN 1

The model requests `check_capacity` for `us-east-1`.

2 HOST EXECUTION

`Runner` validates the arguments. The host reads 0 slots east and 8 west.

3 MODEL TURN 2

The model requests `provision_gpu` for the permitted fallback.

4 HOST EXECUTION

The application approves the pending mutation and reuses one idempotency key.

5 CLOUD RESULT

The host creates `srv-a7` once and `Runner` appends the tool result.

6 MODEL TURN 3

The model returns JSON. `Runner` validates it and returns `DeploymentReport`.

`Runner` carried state between turns. The host controlled validation, approval, and the cloud action.

Section 4: Tools and data contracts

See how `FunctionTools`, `Pydantic` schemas, failure policies, and `output_type` constrain the data exchanged with the model.

Tool type determines execution location and operator responsibility.

FUNCTION TOOL

EXECUTES IN

Your application process

The SDK validates arguments and calls a Python implementation you provide.

`check_capacity`

HOSTED TOOL

EXECUTES IN

OpenAI Platform

The model can use a Platform-operated capability such as web or file search.

`WebSearchTool`

MCP TOOL

EXECUTES IN

An MCP server

The SDK connects to tools exposed through the Model Context Protocol.

`internal catalog`

AGENT AS TOOL

EXECUTES IN

Another agent run

A coordinating agent invokes a specialist agent through a tool interface.

`research_agent`

The GPU deployment example implements local `FunctionTools`. The other columns show additional SDK tool categories.

The tool category determines where code executes and who operates it.

`@function_tool` turns a Python callable into an SDK tool object.

PYTHON SOURCE YOU WRITE

ABRIDGED APPLICATION CODE · cloud_tools.py

```python
@function_tool
def check_capacity(region: str, gpu_count: int) -> str:
    """Read available A100 capacity.

    Args:
        region: Cloud region to inspect.
        gpu_count: Number of GPUs requested.
    """
    return capacity_client.read(region, gpu_count)
```

OBJECT THE SDK REGISTERS

conceptual wrapper

```
FunctionTool(
  name="check_capacity",
  description="Read available A100 capacity.",
  params_json_schema={...},
  on_invoke_tool=<generated adapter>,
)
```

GENERATED ADAPTER

Receives validated arguments, calls the Python function, and converts its return value into a tool result.

After decoration, `check_capacity` is a `FunctionTool`. Current SDK docs also use the shorter `@tool` alias.

The model sees tool metadata. The generated adapter invokes Python in your application.

The FunctionTool wrapper automates the following

1 Describe — Use the function name and docstring as model-visible tool metadata.

2 Build schema — Create a `Pydantic` parameter model and derive JSON Schema.

3 Validate — Parse the model's JSON arguments and enforce types and constraints.

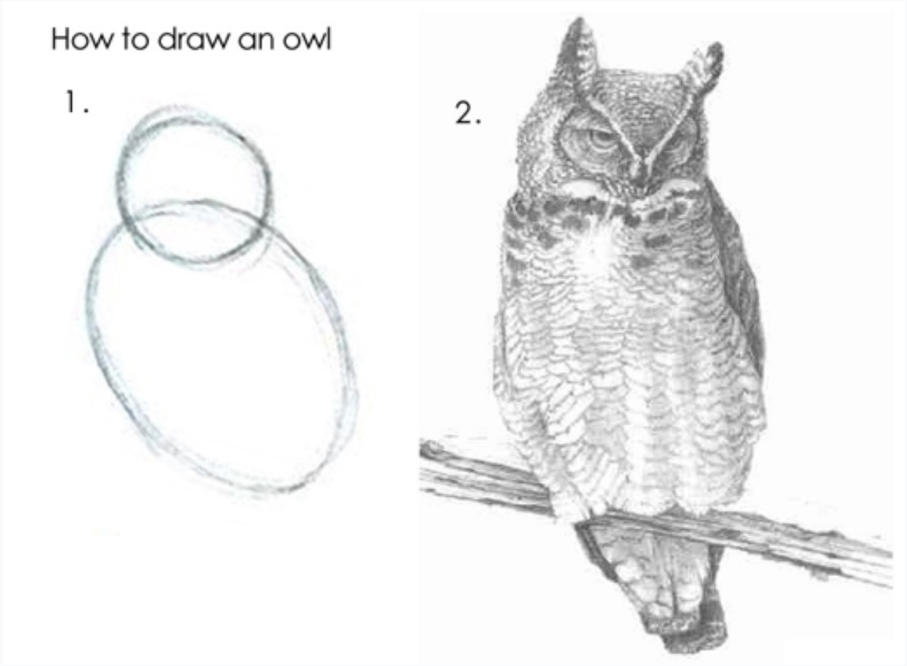
4 Invoke — Convert validated values to Python arguments and call sync or async code.

5 Handle failures — Use the default or custom failure formatter, or propagate an exception.

6 Observe — Emit tool items, streaming events, and a function span in the trace.

The wrapper handles validation and invocation. Application code owns authorization and cloud rules.

The docstring tells the model when and how to invoke a tool



1. ‘Checks servers’ 2. Infer missing constraints

THE SDK PLACES THIS TEXT IN THE MODEL-VISIBLE TOOL DEFINITION

VAGUE DOCSTRING

ABRIDGED EXAMPLE · description

```
"""Checks servers."""
```

The model is missing the resource, provider, valid regions, and the condition that should trigger the fallback.

USEFUL DOCSTRING

ABRIDGED EXAMPLE · description

```
"""Check A100 capacity for an AWS region,
for example 'us-east-1'."""
```

This description names the resource, location, and a decision-relevant example. Type hints define field types on the next slide.

A useful docstring supplies decision context that the Python function name cannot express.

Pydantic converts type hints into a tool-input contract.

1 PYTHON SIGNATURE

ABRIDGED EXAMPLE · function signature

```
def check_capacity(region: str, gpu_count: int):
```

2 GENERATED PYDANTIC MODEL

conceptual generated model

```
class CheckCapacityArgs(BaseModel):
    region: str
    gpu_count: int
```

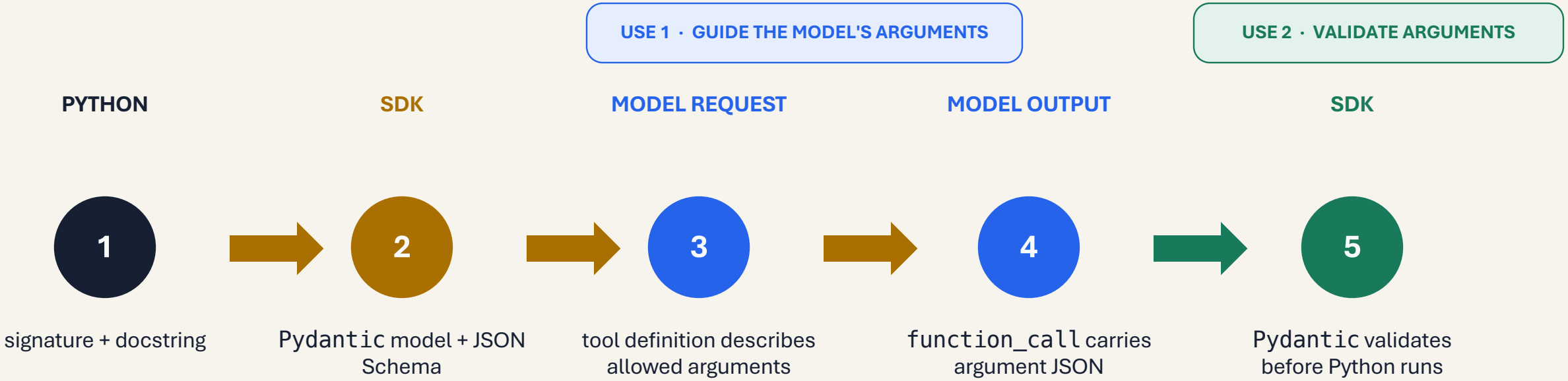
3 JSON SCHEMA SENT WITH THE MODEL REQUEST

ABRIDGED EXAMPLE · params_json_schema excerpt

```
{"type":"object", "properties":{
  "region": {"type":"string"},
  "gpu_count": {"type":"integer"}
}, "required":["region","gpu_count"]}
```

This reuses Session 1 Pydantic concepts: model class, JSON Schema, and validation.

One tool schema guides the model and validates returned arguments



The schema guides the model and validates the returned arguments. It does not prove that a valid string names a real region or that the business action is safe.

Invalid arguments are rejected before the user function runs.

ABRIDGED EXAMPLE · model function_call

```
{
  "name": "check_capacity",
  "arguments": {
    "region": "us-west-2", "gpu_count": "two"
  }
}
```



PYDANTIC VALIDATION ERROR

```
gpu_count
Input should be a valid integer
```

check_capacity(...) is not called

Decorator choice: omit `failure_error_function` to use the SDK's default model-visible error, or set `failure_error_function=None` to propagate the exception to application code.

A strict schema reduces malformed calls, but the chosen failure policy determines who handles them.

Tool input and final output are separate contracts.

TOOL-INPUT CONTRACT

Purpose: validate arguments before one Python function runs.

ABRIDGED EXAMPLE · created by @function_tool

```
check_capacity(
  region: str,
  gpu_count: int)
```

Checked by the `FunctionTool` wrapper with `Pydantic` before the function call.

FINAL-OUTPUT CONTRACT

Purpose: require the completed run to return a value your application can consume.

ABRIDGED EXAMPLE · canonical final-output type

```
class DeploymentReport(BaseModel):
    instance_id: str | None
    requested_region: str
    region: str | None
    status: Literal["ready", "failed"]

cloud_agent = Agent(
    output_type=DeploymentReport)
```

The model uses structured output. The SDK validates and parses the returned JSON into this type.

Both use `Pydantic`, but they protect different data at different times.

`failure_error_function` controls who receives a tool failure.

ABRIDGED APPLICATION CODE · cloud_tools.py

```python
def explain_failure(ctx, error: Exception) -> str:
    return "Capacity check failed. Do not call provision_gpu."

@function_tool(
    failure_error_function=explain_failure
)
def check_capacity(region: str, gpu_count: int) -> str:
    return capacity_client.read(region, gpu_count)
```

CALLBACK

Replaces the internal exception with a domain-safe instruction placed in the model's next input.

CONFIGURATION

The decorator uses this callback only when validation or tool execution fails.

MODEL sees the returned message. APPLICATION still sees trace and logs.

The callback converts an exception into a tool result that can enter the next model turn.

Choose a failure policy based on who should decide next.

DECORATOR SETTING	WHAT HAPPENS	WHEN IT FITS
`@function_tool`	Omitting the argument uses the SDK's default generic `tool-error` message.	Useful when the model may safely retry, choose another tool, or explain failure.

The default policy gives the model a generic error result so it can choose the next step.

Choose a failure policy based on who should decide next.

DECORATOR SETTING	WHAT HAPPENS	WHEN IT FITS
`@function_tool`	Omitting the argument uses the SDK's default generic `tool-error` message.	Useful when the model may safely retry, choose another tool, or explain failure.
`@function_tool( failure_error_function= explain_failure)`	Your callback sends a domain-specific, redacted message to the model.	Useful when the model needs a precise allowed next action without internal details.

A custom callback gives the model specific, redacted guidance without exposing internal details.

Choose a failure policy based on who should decide next.

DECORATOR SETTING	WHAT HAPPENS	WHEN IT FITS
`@function_tool`	Omitting the argument uses the SDK's default generic `tool-error` message.	Useful when the model may safely retry, choose another tool, or explain failure.

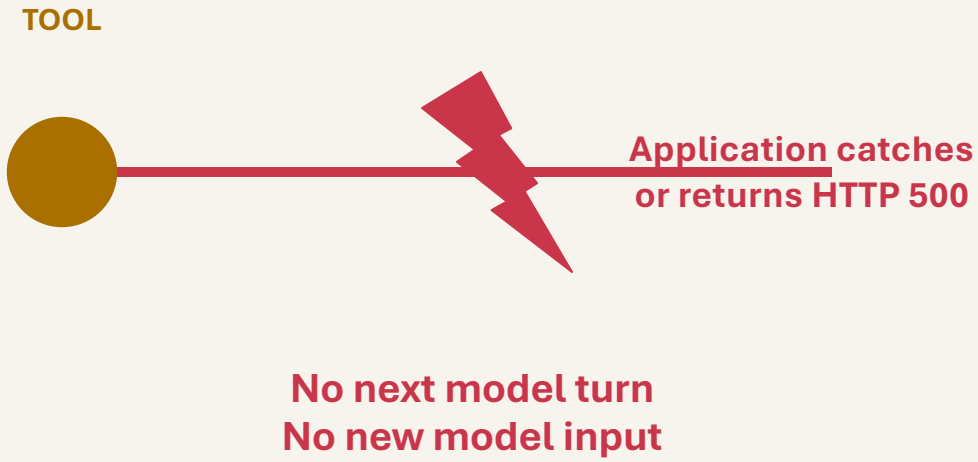
`@function_tool(`
`  failure_error_function=`
`  explain_failure)`
	Your callback sends a domain-specific, redacted message to the model.	Useful when the model needs a precise allowed next action without internal details.

`@function_tool(` `  failure_error_function=None)`	No model-visible error is created. The exception propagates to the application.	Useful when the run must stop and application code must alert, compensate, or decide.

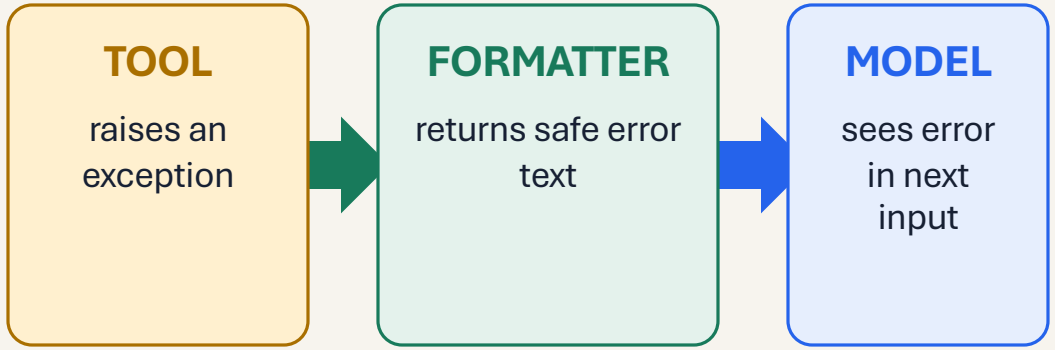
Passing None ends model recovery and propagates the exception to application code.

A propagated Python exception is not visible to the model.

`FAILURE_ERROR_FUNCTION=NONE`



DEFAULT OR CUSTOM FORMATTER



The model can react only to information included in a model request. Python stack unwinding does not automatically enter model context.

To support model recovery, catch or format the failure and place it in the tool result.

A suggested region is not evidence that capacity exists there.

ADVICE-ONLY RESULT

CONCEPTUAL TOOL OUTPUT · advice only

```
{
  "available": false,
  "suggested_region": "us-west-2"
}
```

Open question: Was `us-west-2` measured, or merely recommended?

EVIDENCE-BEARING RESULT

PROTOCOL EXAMPLE · canonical CapacityReport

```
{
  "requested_region": "us-east-1",
  "available": false,
  "available_slots": 0,
  "recommended_region": "us-west-2",
  "recommended_available_slots": 8
}
```

The fallback recommendation now includes the measured slot count that supports it.

The application can audit why the model selected the permitted fallback.

A retry is harmless for a read but can duplicate a mutation.

READ-ONLY CAPACITY CHECK

Call 1	Call 2	Cloud state
read `us-west-2` returns 8 slots	same read repeats returns 8 slots	no server created capacity unchanged

MUTATING PROVISION WITHOUT REPLAY PROTECTION

Attempt 1	Retry	Cloud state
creates `srv-a7` reply is lost	same request runs creates `srv-b9`	2 servers exist only 1 was wanted

The retry may come from the application, a network recovery path, or another model request. The tool must still avoid duplicate state.

Idempotency means repeating the same logical mutation produces no additional side effect.

An idempotency key identifies a repeated deployment request.

ABRIDGED EXAMPLE · first tool call

```
provision_gpu(
  region="us-west-2",
  gpu_count=1,
  idempotency_key="deploy-course-42"
)
```

DEPLOYMENT REQUEST LEDGER

`deploy-course-42` → `srv-a7` → **ready**

The first accepted request creates the server and records the result.

ABRIDGED EXAMPLE · retry with the same key

```
provision_gpu(region="us-west-2", gpu_count=1,
  idempotency_key="deploy-course-42")
# returns existing srv-a7
```

REPLAY DETECTED

Return `srv-a7` from the ledger.
Do not create `srv-b9`.

Approval controls whether a mutation may run; idempotency controls whether a replay creates more state.

What `output_type=DeploymentReport` changes in the completed run

ABRIDGED APPLICATION CODE · contract excerpt

```python
class DeploymentReport(BaseModel):
    instance_id: str | None
    requested_region: str
    region: str | None
    status: Literal["ready", "failed"]

cloud_agent = Agent(
    name="Cloud deployer",
    tools=[check_capacity, provision_gpu],
    output_type=DeploymentReport,
)
```

WHAT THE SDK DOES AFTER OUTPUT_TYPE IS SET

1 CREATE

`Pydantic TypeAdapter` builds a schema and validator for the `DeploymentReport` Python type.

2 REQUEST

The model is told to use structured output for that schema.

3 CHECK

The SDK validates the model's JSON against the output type.

4 RETURN

`result.final_output` is a `DeploymentReport` value.

This contract governs the completed run; it is not the schema for any individual tool call.

“Done! Your server is ready.” cannot satisfy `DeploymentReport`.

REJECTED FINAL VALUE

Done! Your server is ready.

Reason: no `instance_id`, region, status, or `requested_region` fields.

ABRIDGED EXAMPLE · accepted structured value

```
{
  "instance_id": "srv-a7",
  "region": "us-west-2",
  "status": "ready",
  "requested_region": "us-east-1"
}
```

WHERE THE VALIDATED TYPE IS USED

- `FastAPI` (Python web API) response
- Deployment database record
- Operations dashboard fields
- Next workflow decision

A typed final value connects the model run to ordinary application code.

Invalid model output: raise an error or return a typed failure

The agent promises to return a `DeploymentReport`. If the model's final response cannot be converted into that type, the application can either let the run fail or substitute a valid failure report.

DEFAULT BEHAVIOR

VALIDATION FAILS

`Runner` cannot construct `DeploymentReport` from the model output.

`ModelBehaviorError`

The application catches or reports the error.

APPLICATION-SPECIFIC FALLBACK

ABRIDGED EXAMPLE · application handler returns a typed failure

```
return DeploymentReport(
  instance_id=None,
  requested_region="us-east-1",
  region=None,
  status="failed")
```

SDK VALIDATES THE FALLBACK

It must satisfy the same `output_type`.

Section 5: Observing and diagnosing a run

Use streaming to report live progress and traces to reconstruct the operations that produced the final result.

Streaming and tracing expose one run for different purposes.

STREAM EVENTS

Observed while the run is happening

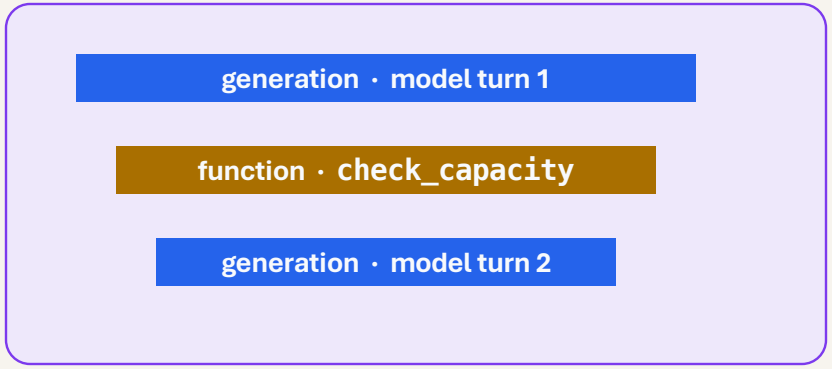
ABRIDGED EXAMPLE · RunItemStreamEvent.name

```
tool_called
tool_output
message_output_created
```

Use them to update a terminal, progress panel, or user interface before the run finishes.

TRACE

Recorded as a timed diagnostic history



Use it after or during a run to inspect model calls, tool calls, timing, errors, and relationships.

Stream events support live experience; traces support diagnosis and monitoring.

A streamed run reports completed items as they happen. 1/5

EXAMPLE UI UPDATES DERIVED FROM RUNITEMSTREAMEVENT

CONCEPTUAL TRACE · deployment progress

```
tool_called   check_capacity(region=us-east-1, gpu_count=1)
```

1 2 3 4 5

Each new event lets the interface explain what the run is doing before it finishes.

A streamed run reports completed items as they happen. 2/5

EXAMPLE UI UPDATES DERIVED FROM RUNITEMSTREAMEVENT

CONCEPTUAL TRACE · deployment progress

```
tool_called   check_capacity(region=us-east-1, gpu_count=1)

tool_output   available=false; available_slots=0; fallback has 8
```

1 2 3 4 5

Each new event lets the interface explain what the run is doing before it finishes.

A streamed run reports completed items as they happen. 3/5

EXAMPLE UI UPDATES DERIVED FROM RUNITEMSTREAMEVENT

CONCEPTUAL TRACE · deployment progress

```
tool_called   check_capacity(region=us-east-1, gpu_count=1)
tool_output   available=false; available_slots=0; fallback has 8
tool_called   provision_gpu(region=us-west-2, gpu_count=1, key=deploy-42)
```

1 2 3 4 5

Each new event lets the interface explain what the run is doing before it finishes.

A streamed run reports completed items as they happen. 4/5

EXAMPLE UI UPDATES DERIVED FROM RUNITEMSTREAMEVENT

CONCEPTUAL TRACE · deployment progress

```
tool_called   check_capacity(region=us-east-1, gpu_count=1)
tool_output   available=false; available_slots=0; fallback has 8
tool_called   provision_gpu(region=us-west-2, gpu_count=1, key=deploy-42)
tool_output   instance_id=srv-a7, region=us-west-2, status=ready
```

1 2 3 4 5

Each new event lets the interface explain what the run is doing before it finishes.

A streamed run reports completed items as they happen. 5/5

EXAMPLE UI UPDATES DERIVED FROM RUNITEMSTREAMEVENT

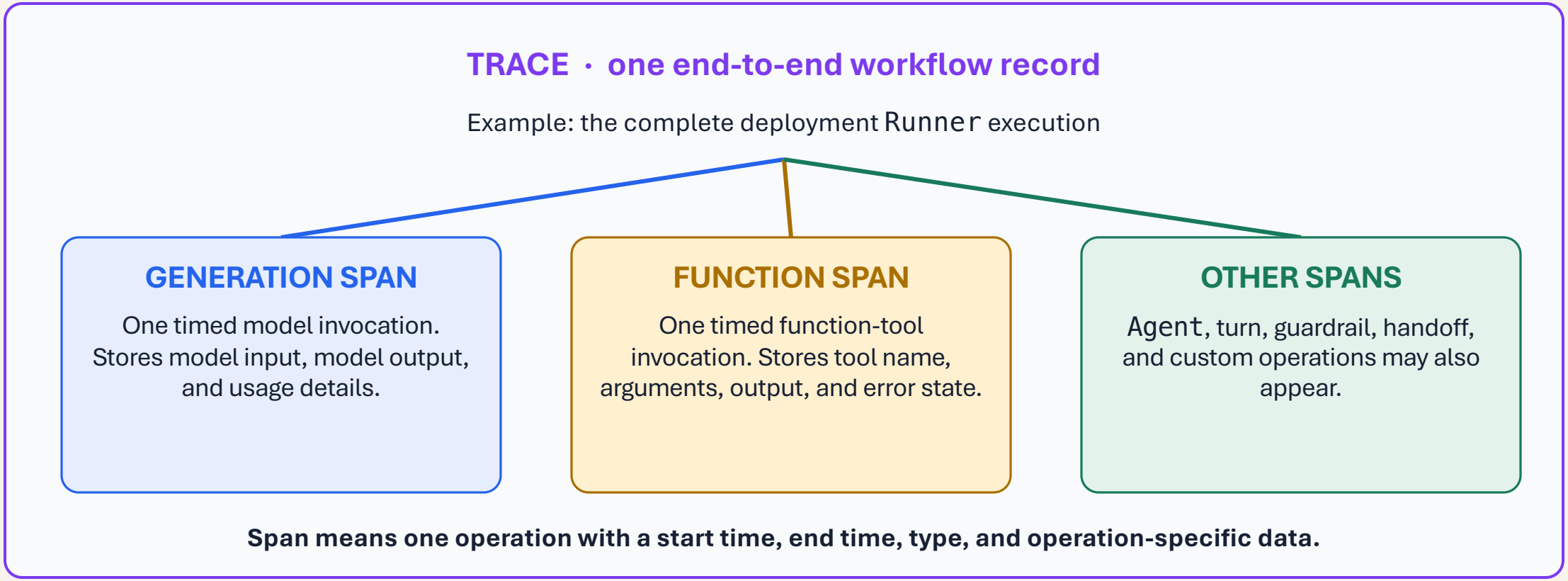
CONCEPTUAL TRACE · deployment progress

```
tool_called   check_capacity(region=us-east-1, gpu_count=1)
tool_output   available=false; available_slots=0; fallback has 8
tool_called   provision_gpu(region=us-west-2, gpu_count=1, key=deploy-42)
tool_output   instance_id=srv-a7, region=us-west-2, status=ready
message_output_created   DeploymentReport(requested=us-east-1, actual=us-west-2)
```



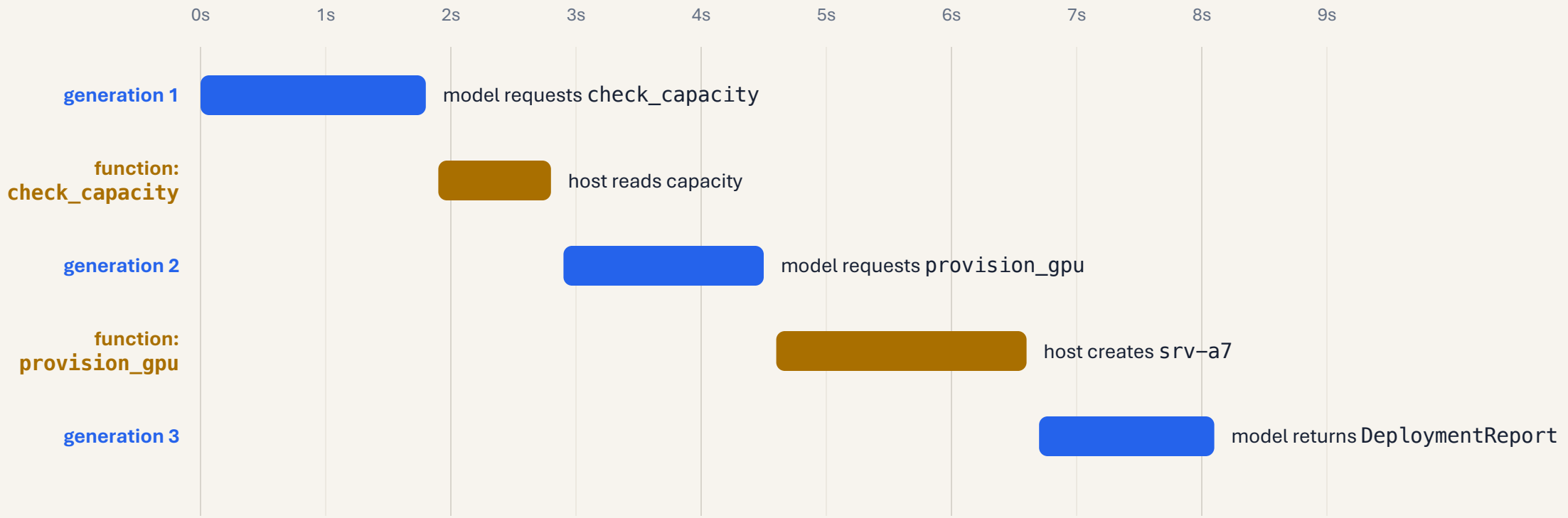
The final message event appears only after the third model invocation produces validated output.

A trace contains timed spans for the operations in one workflow.



The deployment trace shows three generations and two function calls.

ILLUSTRATIVE TIMING FOR: DEPLOY ONE `A100` WITH FALLBACK REGION



Generation means the model generated one response. Function means the host invoked one registered `FunctionTool`.

The trace lets you locate which model invocation or function call first produced the wrong state.

A generation span records one model invocation and its result.

GENERATION SPAN · model turn 2

In this deck, one model turn is one model invocation. That invocation produces one generation span in the trace.

CONCEPTUAL TRACE · model input summary

```
instructions: Check capacity before provisioning.
user: Deploy one A100 in us-east-1.
tools: check_capacity, provision_gpu
tool output: available=false; available_slots=0
tool output: us-west-2 has 8 observed slots
```



CONCEPTUAL TRACE · model output summary

```
type: function_call
name: provision_gpu
arguments:
  region: us-west-2, gpu_count: 1
  key: deploy-42
call_id: call_provision
```

This information appears in the trace viewer. It is not code that you paste into a system prompt.

Inspect a generation span when the model chose the wrong tool, arguments, or final answer.

Traces may contain sensitive model and tool data, so define redaction, access, and retention policies.

WHAT MAY BE RECORDED

Model inputs and outputs, tool arguments and results, timing, errors, usage, and span relationships.

WHERE IT CAN GO

The default exporter can send traces to the `OpenAI` dashboard. A custom processor can send them to your own observability system.

WHAT TO CONFIGURE

Set `trace_include_sensitive_data=False` when payloads should be excluded. Tracing is unavailable under Zero Data Retention.

Before enabling production tracing, answer: which fields may contain secrets or personal data, who can read the trace store, how long traces are retained, and which data must be redacted?

Tracing adds evidence, but it also creates a diagnostic data record that must be governed.

call_id: The model creates the request; the host creates the result.

MODEL OUTPUT FROM ONE GENERATION

ABRIDGED EXAMPLE · function_call item

```
{
  "type": "function_call",
  "name": "check_capacity",
  "arguments": "{\"region\":\"us-east-1\",\"gpu_count\":1}",
  "call_id": "call_cap"
}
```



HOST OUTPUT AFTER LOCAL EXECUTION

PROTOCOL EXAMPLE · function_call_output item

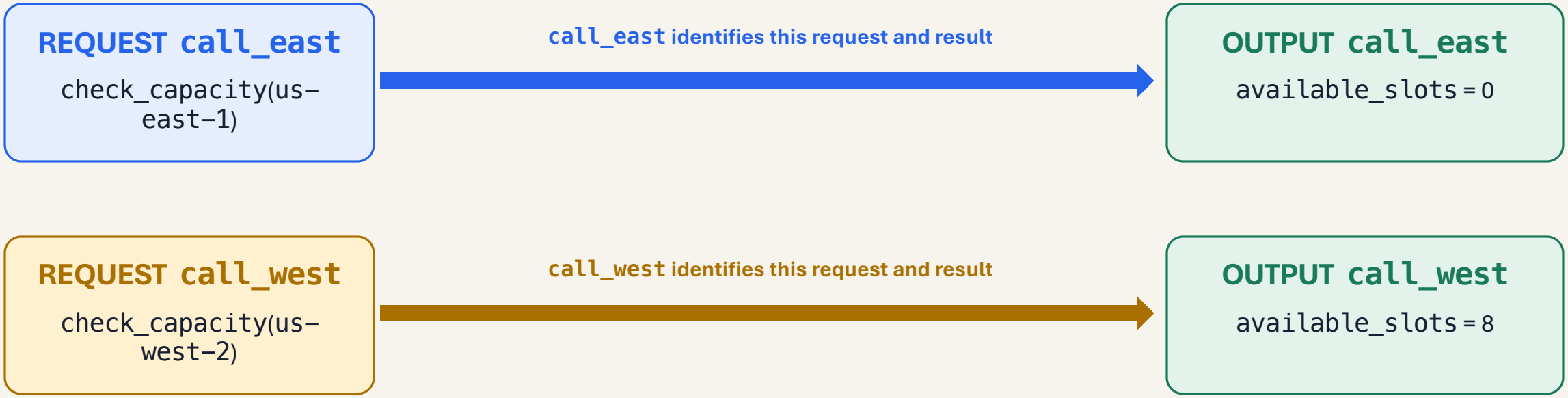
```
{
  "type": "function_call_output",
  "call_id": "call_cap",
  "output": "{\"available\":false,...}"
}
```

The tool definition schema was sent with the model request. The returned arguments are parsed and validated before Python runs; the tool result is appended before the next model invocation.

This JSON is protocol data moving across the model interface, not Python source code.

`call_id` matches each concurrent request to its result

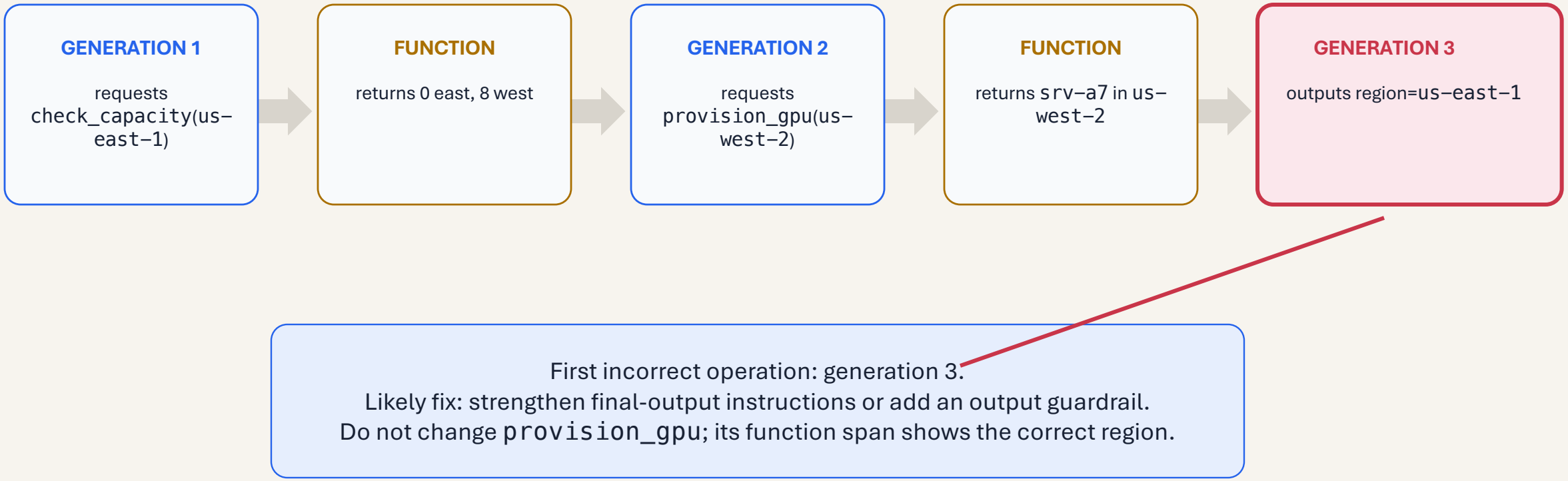
TIME MOVES LEFT TO RIGHT; RESULTS MAY COMPLETE OUT OF ORDER



Without this correlation key, parallel calls to the same tool could not be reliably matched to their results. The key also makes traces and tests easier to follow.

Find the first wrong value before deciding what to fix

HYPOTHETICAL FAILURE: THE FINAL REPORT SAYS US-EAST-1 EVEN THOUGH THE SERVER IS IN US-WEST-2



The first incorrect operation identifies whether to change instructions, output checks, or tool code.

Section 6: Testing each layer

Use `pytest` for Python functions, `ScriptedModel` for `Runner` workflows, and evaluations that call a real model.

Three verification methods answer different questions

3 MODEL EVALUATION

Does a real model choose useful actions and produce good results across representative cases?

2 RUNNER WORKFLOW TEST

Does the SDK route tool calls, validate arguments, append outputs, and stop correctly?

`ScriptedModel`; real `Runner`; real `FunctionTool` pipeline

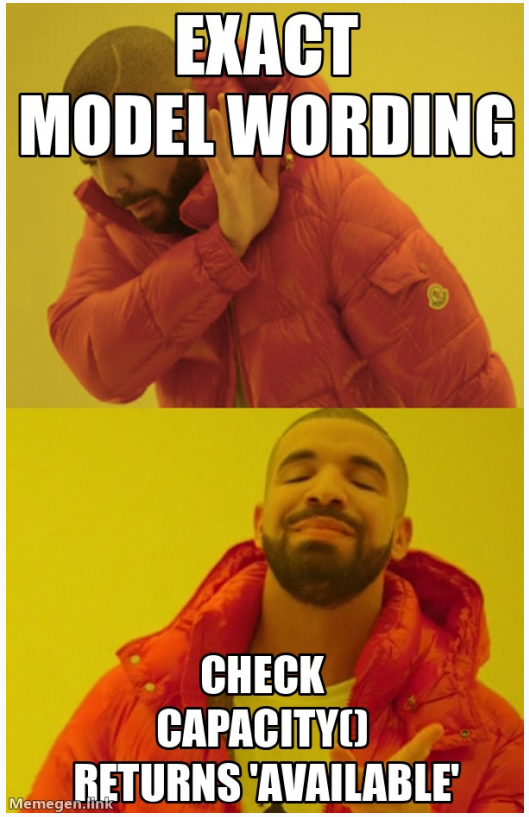
1 PYTHON UNIT TEST

Does our deterministic tool implementation return the correct data for controlled inputs?

`pytest`; real function; fake external cloud client if needed

First identify the question; then choose the narrowest test that can answer it.

Test output fields and deterministic application behavior, not exact model wording.



BRITTLE ASSERTION ABOUT MODEL WORDING

ABRIDGED EXAMPLE · two acceptable sentences, one failed equality test

```
expected = "Deployment successful!"
actual   = "Your server is ready."
assert actual == expected  # fails
```

The model can express the same outcome with different words. Exact string equality can therefore report a failure even when the deployment is correct.

STABLE ASSERTIONS ABOUT APPLICATION BEHAVIOR

ABRIDGED TEST CODE · stable contract assertions

```
assert report.instance_id == "srv-a7"
assert report.status == "ready"
assert check_capacity_impl(...).available is True
```

`output_type` validates the report fields.
`check_capacity_impl` returns deterministic Python data.

Use field assertions for application contracts. Use evaluations when model wording quality is the question.

Test the output contract separately from the workflow

1 CONTRACT VALIDATION TEST

CONTROLLED INVALID FINAL OUTPUT

Omit `instance_id` or use a status outside the Literal values.

ASSERT DETECTION / FALLBACK

Expect `ModelBehaviorError` or the configured typed fallback.

2 ORCHESTRATION WORKFLOW TEST

CONTROLLED VALID MODEL SCRIPT

Request `check_capacity`, consume its result, then return a valid `DeploymentReport`.

ASSERT ROUTING / DATA / STOPPING

Check calls, `function_call_output`, final fields, and script completion.

One test proves that bad final data is rejected; another proves that a valid run is orchestrated correctly.

A unit test runs real tool code with controlled cloud data.

ABRIDGED TEST CODE · test_cloud_tools.py

```python
def test_capacity_available():
    fake_cloud = FakeCapacityClient(slots={"us-west-2": 8})

    result = check_capacity_impl(
        region="us-west-2", gpu_count=1, client=fake_cloud
    )
    assert result.available is True
    assert result.available_slots == 8
```

ARRANGE
The fake cloud client supplies known external data.

ACT
The real `check_capacity_impl` function runs.

ASSERT
The test checks exact structured fields.

Real: tool implementation. Fake: cloud dependency. Absent: model and `Runner`.

Mock the external service only when the unit test needs controlled data or no network access.

Zero capacity is expected data; a service outage is a failure.

ABRIDGED TEST CODE · test_cloud_tools.py

```python
def test_capacity_unavailable():
    fake_cloud = FakeCapacityClient(slots={"us-east-1": 0})
    result = check_capacity_impl(
        region="us-east-1", gpu_count=1, client=fake_cloud
    )
    assert result.available is False
    assert result.available_slots == 0
```

EXPECTED CONDITION

Return structured data the model or application can reason about. Do not raise merely because capacity is zero.

UNEXPECTED FAILURE

A timeout, authentication failure, or corrupt response may raise. Configure whether the model or application receives it.

Real: tool implementation. Fake: cloud dependency. Absent: model and `Runner`.

Return ordinary business outcomes as data; reserve exceptions for failures of the operation itself.

Each model invocation consumes one `ScriptedModel` response

`ScriptedModel` replaces the network model. `Runner`, `FunctionTool` validation, routing, approval, and tool execution remain real.

MODEL CALL 1	`function_call(check_capacity)`	`Runner` appends `CapacityReport`
MODEL CALL 2	`function_call(provision_gpu)`	Application approves; tool creates one server
MODEL CALL 3	`assistant_message(DeploymentReport)`	`Runner` validates and stops

CONCEPTUAL TEST CODE · three-response script shape

```
model = ScriptedModel([[function_call(...)], [function_call(...)], [assistant_message(...)]])
```

Two separate `Runner.run` calls consume two queued responses

RUNNABLE TEST CODE · two separate Runner executions

```python
from agents import Agent, Runner
from agents.testing import ScriptedModel, assistant_message

async def test_agent_run():
    model = ScriptedModel()
    model.enqueue([assistant_message("Hello! Simulated response.")])
    model.enqueue([assistant_message("The task is complete.")])

    agent = Agent(name="test_agent", model=model)

    first = await Runner.run(agent, "Hi there.")
    second = await Runner.run(agent, "Did you finish?")

    assert first.final_output == "Hello! Simulated response."
    assert second.final_output == "The task is complete."
    model.assert_complete()
```

1. IMPORT

`assistant_message` builds the model-response object that enqueue expects.

2. SCRIPT

Each enqueue call adds one complete model response. Two responses are queued for two separate `Runner.run` calls.

3. INJECT

`model=model` makes the `Agent` use the in-memory fake instead of a network model.

4. RUN AND CHECK

Field assertions verify results. `assert_complete` separately proves that no queued response was left unused.

Script the model dependency, run the real orchestration code, and assert both outputs and script consumption.

The deployment workflow test requires three model calls and one mutation

1 CAPACITY

Model call 1 requests `check_capacity`. `Runner` appends available=false and the measured fallback slots.

2 PROVISION

Model call 2 requests `provision_gpu`. The application approves the pending call; the ledger gains one entry.

3 FINAL REPORT

Model call 3 consumes the provision result and returns the canonical `DeploymentReport`.

TEST EVIDENCE · stable assertions

```
assert len(model.calls) == 3
assert len(CREATED_BY_KEY) == 1
assert result.final_output.requested_region == "us-east-1"
assert result.final_output.region == "us-west-2"
model.assert_complete()
```

A passing test now requires the same capacity, approval, mutation, and final-output path taught earlier.

A turn-limit test proves the run stops after repeated model calls.

ABRIDGED TEST CODE · imports and request omitted

```
model = ScriptedModel([
  [function_call("check_capacity",
    {"region": "us-east-1", "gpu_count": 1}, call_id="c1")],
  [function_call("check_capacity",
    {"region": "us-east-1", "gpu_count": 1}, call_id="c2")],
])
test_agent = cloud_agent.clone(model=model)

with pytest.raises(MaxTurnsExceeded):
    await Runner.run(
        test_agent, request, max_turns=2,
        run_config=RunConfig(tracing_disabled=True),
    )
```

WHAT THE SCRIPT PROVES

MODEL TURN 1

Requests a tool; no final output.

MODEL TURN 2

Requests a tool again; still no final output.

STOP

`Runner` raises `MaxTurnsExceeded` instead of attempting model turn 3.

Purpose: cap latency, model cost, and repeated tool opportunities when the model never reaches final output.

The test is proving that the safety limit works when a model keeps requesting tools instead of finishing

Section 7: Complete application

Combine the agent configuration, tools, output contract, `Runner` call, and tests in one implementation, then apply the design in the lab.

The application combines typed tools, policy, and Runner

APPLICATION CODE · abridged integration

```python
class DeploymentReport(BaseModel):
    instance_id: str | None
    requested_region: str
    region: str | None
    status: Literal["ready", "failed"]

@function_tool
def check_capacity(region: str, gpu_count: int) -> CapacityReport:
    return check_capacity_impl(region, gpu_count)

@function_tool
def provision_gpu(region: str, gpu_count: int, idempotency_key: str) -> ProvisionResult:
    return provision_impl(region, gpu_count, idempotency_key)

cloud_agent = Agent(
  instructions="Check capacity before provisioning.",
  tools=[check_capacity, provision_gpu],
  output_type=DeploymentReport,
)

result = await Runner.run(cloud_agent, request, max_turns=4)
report = result.final_output
```

PYDANTIC
Defines the final Python type consumed by the application.

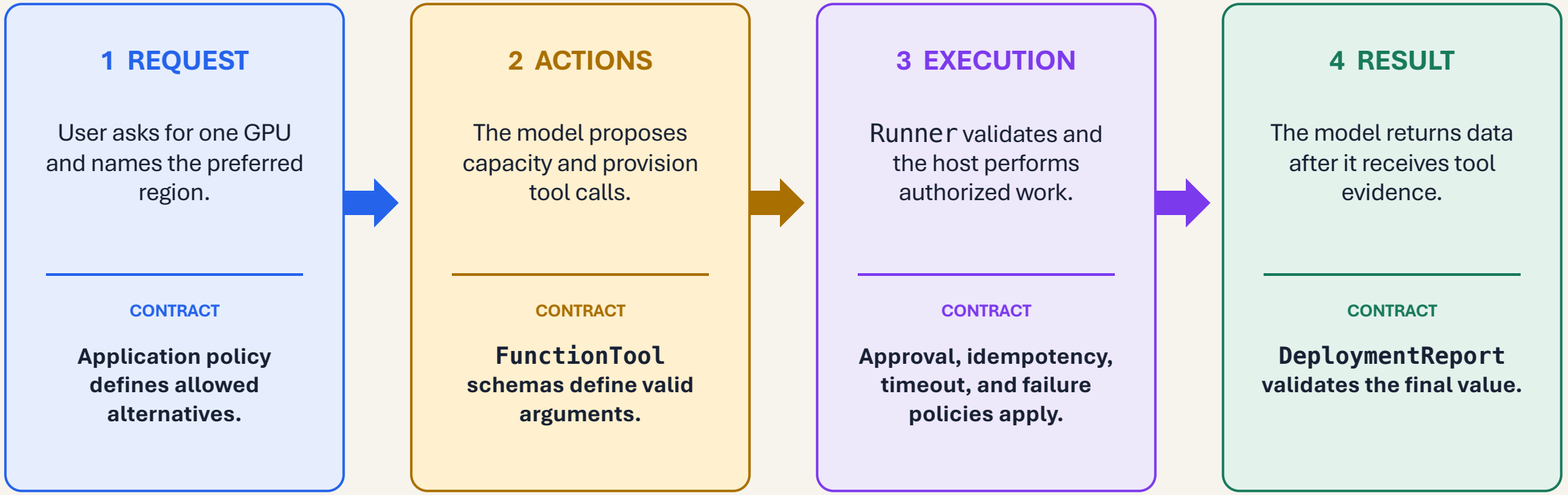
FUNCTION TOOLS
Expose typed, documented host functions through generated tool definitions.

AGENT
Collects instructions, allowed tools, and the final-output contract.

RUNNER
Executes the model and tool loop, then returns a run result.

The SDK supplies orchestration; the application's functions still perform and secure the real work.

One request moves through four explicit contracts.



Streaming and tracing expose these operations; unit and workflow tests verify the deterministic parts.

The SDK connects the contracts; your application supplies the domain rules and real side effects.

Lab: one safe, inspectable, and testable agent run

1

DEFINE

Write one exact user request and one `Pydantic` final-output type.

2

WRAP

Create at least one `@function_tool` with useful type hints and docstrings.

3

OBSERVE

Run through `Runner` and inspect stream events plus one generation and function span.

4

VERIFY

Write a direct `pytest` unit test and a `ScriptedModel` `Runner` workflow test.

A successful submission explains what the SDK handled, what your application handled, and what each test proves.

Supervise the loop, define the contracts, inspect the operations, and test the stable behavior.

Appendix: Runnable implementation reference

Assemble the `FunctionTools`, `Agent`, `Runner` call, approval flow, and deterministic workflow test into one small project.

Pinned environment and canonical data contracts

RUNNABLE PROJECT FILE · requirements.txt

```
openai-agents==0.22.0
pydantic>=2.12,<3
pytest
pytest-asyncio
```

VERSION NOTE

0.22.0 was the latest published `openai-agents` release checked on 2026-09-04. Recheck the course repository before using this example later.

RUNNABLE APPLICATION CODE · canonical data types

```python
import asyncio
from typing import Literal
from agents import Agent, Runner
from agents.decorators import tool
from pydantic import BaseModel
class CapacityReport(BaseModel):
    requested_region: str
    available: bool
    available_slots: int
    recommended_region: str | None = None
    recommended_available_slots: int | None = None
class ProvisionResult(BaseModel):
    instance_id: str
    region: str
    status: Literal["ready"]
class DeploymentReport(BaseModel):
    instance_id: str | None
    requested_region: str
    region: str | None
    status: Literal["ready", "failed"]
```

Current docs use `@tool` as a shorter alias for `@function_tool`; both create a `FunctionTool` wrapper.

A runnable example starts with a reproducible version and import paths that match that version.

The read-only tool returns measured evidence from deterministic data.

RUNNABLE APPLICATION CODE · deterministic capacity tool

```python
CAPACITY = {"us-east-1": 0, "us-west-2": 8}

@tool
def check_capacity(region: str, gpu_count: int) -> CapacityReport:
    """Read measured A100 slots and name an allowed fallback."""
    west = CAPACITY["us-west-2"]
    slots = CAPACITY.get(region, 0)
    available = slots >= gpu_count
    fallback = None if available else ("us-west-2" if west >= gpu_count else None)
    return CapacityReport(
        requested_region=region,
        available=available,
        available_slots=slots,
        recommended_region=fallback,
        recommended_available_slots=west if fallback else None,
    )
```

Production code replaces CAPACITY with an authenticated cloud or inventory client.

The returned recommendation includes the measured slot count that supports it.

The mutating tool requires approval and prevents duplicate creation.

RUNNABLE APPLICATION CODE · approval-gated mutation

```python
CREATED_BY_KEY: dict[str, ProvisionResult] = {}

@tool(needs_approval=True)
def provision_gpu(region: str, gpu_count: int, idempotency_key: str) -> ProvisionResult:
    """Create one server or return the prior result for this key."""
    if region != "us-west-2":
        raise ValueError("Region is not approved")
    if gpu_count != 1:
        raise ValueError("This lesson permits exactly one GPU")
    if idempotency_key not in CREATED_BY_KEY:
        CREATED_BY_KEY[idempotency_key] = ProvisionResult(
            instance_id="srv-a7",
            region=region,
            status="ready",
        )
    return CREATED_BY_KEY[idempotency_key]
```

Production code would use durable storage so the idempotency record survives restarts.

Approval controls authorization; the idempotency key controls replay safety.

Application policy approves a saved pending call before `Runner` resumes

RUNNABLE APPLICATION CODE · real model run with approval

```python
cloud_agent = Agent(
    name="Cloud deployer",
    instructions="Check capacity; use only us-west-2 as fallback.",
    tools=[check_capacity, provision_gpu],
    output_type=DeploymentReport,
)

async def main() -> None:
    request = "Deploy one A100 in us-east-1; fallback to us-west-2."
    result = await Runner.run(cloud_agent, request, max_turns=4)
    while result.interruptions:
        state = result.to_state()
        for pending in result.interruptions:
            state.approve(pending)  # replace with real policy or UI
        result = await Runner.run(cloud_agent, state)
    print(result.final_output)

asyncio.run(main())
```

The application applies approval policy outside the model, then resumes the saved run state.

Three scripted model responses cover both tools and the final report

RUNNABLE TEST CODE · model provider fixture

```
import pytest
from agents import RunConfig, Runner
from agents.testing import ScriptedModel, assistant_message, function_call
from app import CREATED_BY_KEY, cloud_agent


model = ScriptedModel([
    [function_call("check_capacity",
        {"region": "us-east-1", "gpu_count": 1}, call_id="cap-1")],
    [function_call("provision_gpu",
        {"region": "us-west-2", "gpu_count": 1,
         "idempotency_key": "deploy-42"},
        call_id="provision-1")],
    [assistant_message(
        '{"instance_id":"srv-a7","requested_region":"us-east-1",'
        '"region":"us-west-2",'
        '"status":"ready"}')],
])
```

The workflow test proves the approved mutation occurs exactly once

RUNNABLE TEST CODE · Runner workflow with approval

```python
@pytest.mark.asyncio
async def test_deployment_workflow():
    CREATED_BY_KEY.clear()
    test_agent = cloud_agent.clone(model=model)
    result = await Runner.run(
        test_agent, "Deploy one A100 in us-east-1; fallback to us-west-2.",
        run_config=RunConfig(tracing_disabled=True))

    assert len(result.interruptions) == 1
    state = result.to_state()
    state.approve(result.interruptions[0])
    result = await Runner.run(test_agent, state)

    assert len(model.calls) == 3
    assert len(CREATED_BY_KEY) == 1
    report = result.final_output
    assert report.instance_id == "srv-a7"
    assert (report.requested_region, report.region, report.status) == (
        "us-east-1", "us-west-2", "ready")
    model.assert_complete()
```

PROOF: three model calls, one approved ledger entry, canonical final fields, and no unused scripted response